

< Day1 실습 과제 >

울산_4반_1조_박인우 day1 실습

실습 1. PK = (order_id, prod_id)

order_id	order_dt	cust_id	cust_name	prod_id	prod_name	unit_price	qty
O1001	2024-06-01	C01	김하늘	P10	노트북	1200000	1
O1001	2024-06-01	C01	김하늘	P22	마우스	25000	2
O1002	2024-06-03	C02	이준호	P10	노트북	1200000	1

실습 과제 — 이 표를 3NF로

1. 각 컬럼의 함수 종속 식별

어떤 컬럼 값이 정해졌을 때, 다른 컬럼 값이 항상 하나로 결정되는가?

order_id가 같으면 주문일자와 주문한 고객도 같으므로:

order_id → order_dt, cust_id

고객번호를 알면 고객 이름이 하나로 결정되므로:

cust_id → cust_name

상품번호를 알면 상품명과 단가가 결정되므로:

prod_id → prod_name, unit_price

수량은 주문번호만으로도, 상품번호만으로도 결정할 수 없다. 어떤 주문에서 어떤 상품을 주문했는지를 모두 알아야 하므로:

(order_id, prod_id) → qty

주의 :

다음 관계는 일반적으로 함수 종속으로 보지 않는다.

prod_name → unit_price

같은 상품명을 사용하는 다른 상품이나 옵션이 존재할 수 있고, 상품명이 같더라도 단가가 다를 수 있기 때문이다. 상품을 정확하게 식별하는 기준은 **prod_name**이 아니라 **prod_id**이다.

2. 1NF→2NF→3NF 단계로 분해

1NF : 각 칸에는 하나의 원자값만 들어가고, 상품1·상품2처럼 반복되는 컬럼 묶음이 없어야 한다.

현재 표는 각 셀에 하나의 값만 들어 있고 반복 컬럼도 없으므로 이미 **1NF**를 만족한다.

2NF : 복합 기본키의 일부에만 종속되는 컬럼, 즉 부분 함수 종속을 제거하는 단계이다.

다음 컬럼은 **order_id**만으로 결정된다.

order_id → **order_dt, cust_id, cust_name**

cust_name은 직접적으로는 **cust_id**에 종속되지만, 이 단계에서는 주문과 관련된 정보에 포함해 먼저 분리할 수 있다.

다음 컬럼은 **prod_id**만으로 결정된다.

prod_id → **prod_name, unit_price**

수량만 복합키 전체에 종속된다.

(order_id, prod_id) → **qty**

따라서 **2NF**에서는 다음 세 테이블로 분해한다

```
주문(  
    order_id PK,  
    order_dt,  
    cust_id,  
    cust_name  
)
```

```
상품(  
    prod_id PK,  
    prod_name,  
    unit_price  
)
```

```
주문상세(  
    order_id PK, FK,  
    prod_id PK, FK,  
    qty
```

)

3NF : 기본키가 아닌 컬럼을 거쳐 다른 컬럼이 결정되는 이행 함수 종속을 제거하는 단계이다.

2NF의 주문 테이블에는 다음 관계가 남아 있다.

$order_id \rightarrow cust_id$
 $cust_id \rightarrow cust_name$

즉:

$order_id \rightarrow cust_id \rightarrow cust_name$

이라는 이행 종속이 존재한다.

cust_name은 주문번호의 직접적인 속성이 아니라 고객번호의 속성이므로, 고객 정보를 별도 테이블로 분리한다.

```
고객 (  
    cust_id PK,  
    cust_name  
)  
주문(  
    order_id PK,  
    order_dt,  
    cust_id FK  
)
```

3. 최종 테이블·PK·FK 정리

```
고객(  
    cust_id PK,  
    cust_name  
)
```

```
주문(  
    order_id PK,  
    order_dt,  
    cust_id FK  
)
```

```
상품(  
    prod_id PK,  
    prod_name,  
    unit_price
```

)

주문상세(

order_id PK, FK,

prod_id PK, FK,

qty

)

고객 1 — N 주문

고객 한 명은 여러 번 주문할 수 있지만, 주문 한 건은 한 고객에게 속한다

주문 1 — N 주문상세

주문 한 건에는 여러 상품이 포함될 수 있으므로 주문상세 행이 여러 개 생길 수 있다.

상품 1 — N 주문상세

상품 하나는 여러 주문에서 판매될 수 있으므로 여러 주문상세 행에 등장할 수 있다

따라서 주문과 상품은 원래 다대다 관계이다.

주문 N — M 상품

이를 주문상세 테이블을 이용해 두 개의 일대다 관계로 해소한 것이다.

주문 1 — N 주문상세 N — 1 상품

최종적으로 고객, 주문, 상품, 주문상세의 네 테이블로 분리함으로써 고객명이나 상품 가격이 변경될 때 여러 행을 반복 수정해야 하는 문제를 줄이고, 삽입·수정·삭제 이상 현상을 방지할 수 있다.

부록. 실제 dbeaver 이용해보기

```

● CREATE TABLE order_detail_raw (
  order_id  VARCHAR(10) NOT NULL,
  order_dt  DATE NOT NULL,
  cust_id   VARCHAR(10) NOT NULL,
  cust_name VARCHAR(50) NOT NULL,
  prod_id   VARCHAR(10) NOT NULL,
  prod_name VARCHAR(50) NOT NULL,
  unit_price INTEGER NOT NULL,
  qty       INTEGER NOT NULL,

  PRIMARY KEY (order_id, prod_id)
);

```

```

● INSERT INTO order_detail_raw (
  order_id,
  order_dt,
  cust_id,
  cust_name,
  prod_id,
  prod_name,
  unit_price,
  qty
)
VALUES
  ('O1001', DATE '2024-06-01', 'C01', '김하늘', 'P10', '노트북', 1200000, 1),
  ('O1001', DATE '2024-06-01', 'C01', '김하늘', 'P22', '마우스', 25000, 2),
  ('O1002', DATE '2024-06-03', 'C02', '이준호', 'P10', '노트북', 1200000, 1);

```

	order_id	order_dt	cust_id	cust_name	prod_id	prod_name	unit_price	qty
1	O1001	2024-06-01	C01	김하늘	P10	노트북	1,200,000	1
2	O1001	2024-06-01	C01	김하늘	P22	마우스	25,000	2
3	O1002	2024-06-03	C02	이준호	P10	노트북	1,200,000	1

lab 스키마

```

● CREATE TABLE customer (
  cust_id VARCHAR(10) PRIMARY KEY,
  cust_name VARCHAR(50) NOT NULL
);

● CREATE TABLE product (
  prod_id VARCHAR(10) PRIMARY KEY,
  prod_name VARCHAR(50) NOT NULL,
  unit_price INTEGER NOT NULL CHECK (unit_price >= 0)
);

● CREATE TABLE orders (
  order_id VARCHAR(10) PRIMARY KEY,
  order_dt DATE NOT NULL,
  cust_id VARCHAR(10) NOT NULL,
  FOREIGN KEY (cust_id)
    REFERENCES customer(cust_id)
);

● CREATE TABLE order_item (
  order_id VARCHAR(10) NOT NULL,
  prod_id VARCHAR(10) NOT NULL,
  qty INTEGER NOT NULL CHECK (qty > 0),

  PRIMARY KEY (order_id, prod_id),

  FOREIGN KEY (order_id)
    REFERENCES orders(order_id),

  FOREIGN KEY (prod_id)
    REFERENCES product(prod_id)
);

```

lab	
Tables	
> customer	8K
> order_item	8K
> orders	8K
> product	8K

```

● INSERT INTO lab.customer (cust_id, cust_name)
  SELECT DISTINCT
    cust_id,
    cust_name
  FROM public.order_detail_raw;

● INSERT INTO lab.product (prod_id, prod_name, unit_price)
  SELECT DISTINCT
    prod_id,
    prod_name,
    unit_price
  FROM public.order_detail_raw;

● INSERT INTO lab.orders (order_id, order_dt, cust_id)
  SELECT DISTINCT
    order_id,
    order_dt,
    cust_id
  FROM public.order_detail_raw;

● INSERT INTO lab.order_item (order_id, prod_id, qty)
  SELECT
    order_id,
    prod_id,
    qty
  FROM public.order_detail_raw;

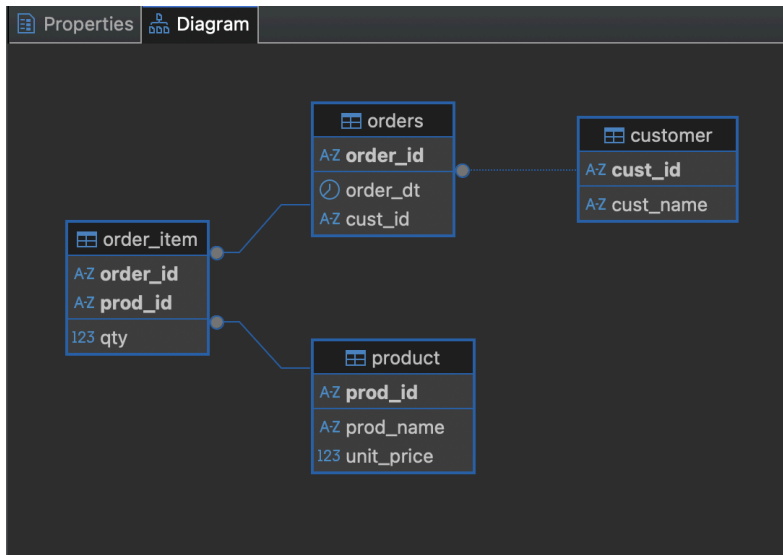
```

	AZ cust_id	AZ cust_name
1	C01	김하늘
2	C02	이준호

	AZ order_id	order_dt	AZ cust_id
1	O1001	2024-06-01	C01
2	O1002	2024-06-03	C02

	AZ order_id	AZ prod_id	123 qty
1	O1001	P10	1
2	O1001	P22	2
3	O1002	P10	1

	AZ prod_id	AZ prod_name	123 unit_price
1	P10	노트북	1,200,000
2	P22	마우스	25,000



실습 2 . 단일 테이블 조회

```

SELECT
  cust_id,
  name,
  birth_ymd,
  grade,
  join_dt,
  COALESCE(region, '미상') AS region, -- NULL → '미상'
  EXTRACT(YEAR FROM AGE(birth_ymd)) AS age,
  CASE
    WHEN EXTRACT(YEAR FROM AGE(birth_ymd)) < 30 THEN '청년'
    WHEN EXTRACT(YEAR FROM AGE(birth_ymd)) < 50 THEN '중년'
    ELSE '장년'
  END AS age_type
FROM customer
WHERE grade IN ('GOLD', 'VIP')
  AND join_dt >= DATE '2020-01-01'
ORDER BY join_dt ASC
LIMIT 15;
  
```

실행 결과

```

select
  cust_id ,
  name,
  birth_ymd,
  grade,
  join_dt ,
  coalesce(region, '미상') as region,
  extract (year from age(birth_ymd)) as age,

  case
    when extract (year from age(birth_ymd)) < 30 then '청년'
    when extract (year from age(birth_ymd)) < 50 then '중년'
    else '장년'
  end as age_type
from customer

where
  grade in ('GOLD', 'VIP')
  and join_dt >= Date '2020-01-01'

order by join_dt asc limit 15;
  
```

	123 cust_id	AZ name	birth_ymd	AZ grade	join_dt	AZ region	123 age	AZ age_type
1	211	고객211	2014-11-13	GOLD	2022-01-05	부산	11	청년
2	198	고객198	1993-09-30	GOLD	2022-01-06	강원	32	중년
3	155	고객155	2016-07-10	GOLD	2022-01-07	강원	10	청년
4	15	고객15	1995-09-23	GOLD	2022-01-12	대전	30	중년
5	320	고객320	1977-09-26	GOLD	2022-01-12	서울	48	중년
6	227	고객227	2010-06-03	GOLD	2022-01-13	부산	16	청년
7	115	고객115	1979-12-15	VIP	2022-01-16	대전	46	중년
8	216	고객216	2006-03-15	GOLD	2022-01-16	강원	20	청년
9	196	고객196	2015-04-12	GOLD	2022-01-29	대전	11	청년
10	430	고객430	2002-08-01	GOLD	2022-01-30	경기	23	청년
11	309	고객309	2015-02-12	GOLD	2022-01-31	대구	11	청년
12	225	고객225	1977-08-05	GOLD	2022-02-01	대구	48	중년
13	243	고객243	2008-02-26	GOLD	2022-02-04	대전	18	청년
14	209	고객209	1996-10-01	GOLD	2022-02-07	경기	29	청년
15	453	고객453	2007-12-31	GOLD	2022-02-09	광주	18	청년

customer 테이블에서 고객 등급이 **GOLD** 또는 **VIP**이고, 가입일이 **2020년 1월 1일** 이후인 고객만 조회하는 쿼리이다.

먼저 **WHERE** 절에서 **grade IN ('GOLD', 'VIP')** 조건으로 두 등급에 해당하는 고객만 남기고, **AND join_dt >= DATE '2020-01-01'** 조건을 추가하여 가입일 조건까지 동시에 만족하는 행만 필터링한다.

조회 결과에는 고객번호, 이름, 생년월일, 등급, 가입일을 표시하며, **COALESCE(region, '미상')**을 사용해 지역 정보가 **NULL**인 경우 조회 결과에서 **'미상'**으로 대체한다. 이 처리는 실제 테이블의 값을 수정하는 것이 아니라 결과 화면에만 적용된다.

또한 **AGE(birth_ymd)**로 현재 날짜와 생년월일의 차이를 계산하고, **EXTRACT(YEAR FROM ...)**을 이용해 만 나이를 구한다. 계산된 나이는 **CASE**문에서 다시 사용하여 **30세** 미만은 **'청년'**, **30세 이상 50세** 미만은 **'중년'**, **50세 이상**은 **'장년'**으로 분류한다

마지막으로 **ORDER BY join_dt ASC**를 사용해 가입일이 오래된 고객부터 정렬하고, **LIMIT 15**를 통해 정렬된 결과 중 최대 **15건**만 출력한다.

즉 이 쿼리는 고객 조건 필터링, **NULL** 처리, 만 나이 계산, 연령대 분류, 가입일 정렬, 출력 건수 제한을 하나의 **SELECT**문 안에서 모두 처리한 조회문이다.

부록 : 주의점, 틀렸던 부분

and join_dt >= Date '2020-01-01' (처음 작성: **and join_dt >='2020-01-01'**),
더 권장되는 작성 방식

case

```

when extract (year from age(birth_ymd)) <30 then '청년'
when extract (year from age(birth_ymd)) <50 then '중년'
else '장년'

```

(처음 작성 :

case

when age<30 then '청년'

when 30 < age <50 then '중년'

else '장년'

)

같은 select절에서 생성한 별칭은 다른데서 참조를 못함.

또한 SQL에서는 $30 < \text{age} < 50$ 같은 연속 비교를 사용할 수 없으므로 $\text{age} > 30$ and $\text{age} < 50$ 처럼 작성해야 한다. 다만 case는 위에서부터 검사하고 처음 참인 조건에서 종료되므로, 첫 번째 조건에서 30세 미만이 이미 처리되어 두 번째 조건은 < 50 만 작성해도 30~49세를 의미한다.